

Client Side - Vulnerabilities

BACKGROUND

Securing a website can be a challenging endeavor. It is not uncommon to see websites which contain vulnerabilities in the code that runs in the browser (we call this "client side"; conversely, the "server side" refers to code running on a server). There are many considerations which need to be accounted for in order to secure a website. This lab exercise will explore a few of those considerations and how to identify or take advantage of vulnerabilities.



DESCRIPTION

This exercise demonstrates how to use a web browser to bypass some client-side restrictions. These steps assume that a Chromium browser is being used, but other browsers such as Firefox are quite similar to use.

REQUIREMENTS

A computer that can run Java. Most modern Windows, Linux and MacOS machines can run Java, as can Google Cloud Shell. This exercise assumes you're running an instance of Linux (either virtualized or other). This exercise will run in Google Cloud Shell. Previous exercises in this chapter walk through the Java and WebGoat installations. A web browser (preferably Chrome) is also required.

PART I: Bypass Front-End Restrictions

It is not uncommon to encounter websites which validate information (such as user input) in the browser. A crafty user is often able to break validation rules before data is sent to the server. This can lead to bad situations on the server. For example, data integrity may be compromised. Or perhaps this sets up a cross site scripting attack. It might be the case that the price of an expensive TV on an e-commerce website might be able to be lowered.

SECTION I: Field restrictions (Step 2)

1. Log into WebGoat
2. From the left navigation bar, select "Client side"
3. From the left navigation bar, select "Bypass front-end restrictions"
4. Click on Step 2 (a red box)

5. The task is to alter what information is sent to the server. There are several ways to accomplish this. Usually attackers will use a special proxy which can intercept and alter requests sent to the server. While that approach is valid, it is not necessary for this exercise. Instead, explore the developer tools. To do this, right click on one of the fields to alter and select "Inspect" from the pop-up menu. This will open the developer tools to the "Elements" tab with the element you clicked on already selected.



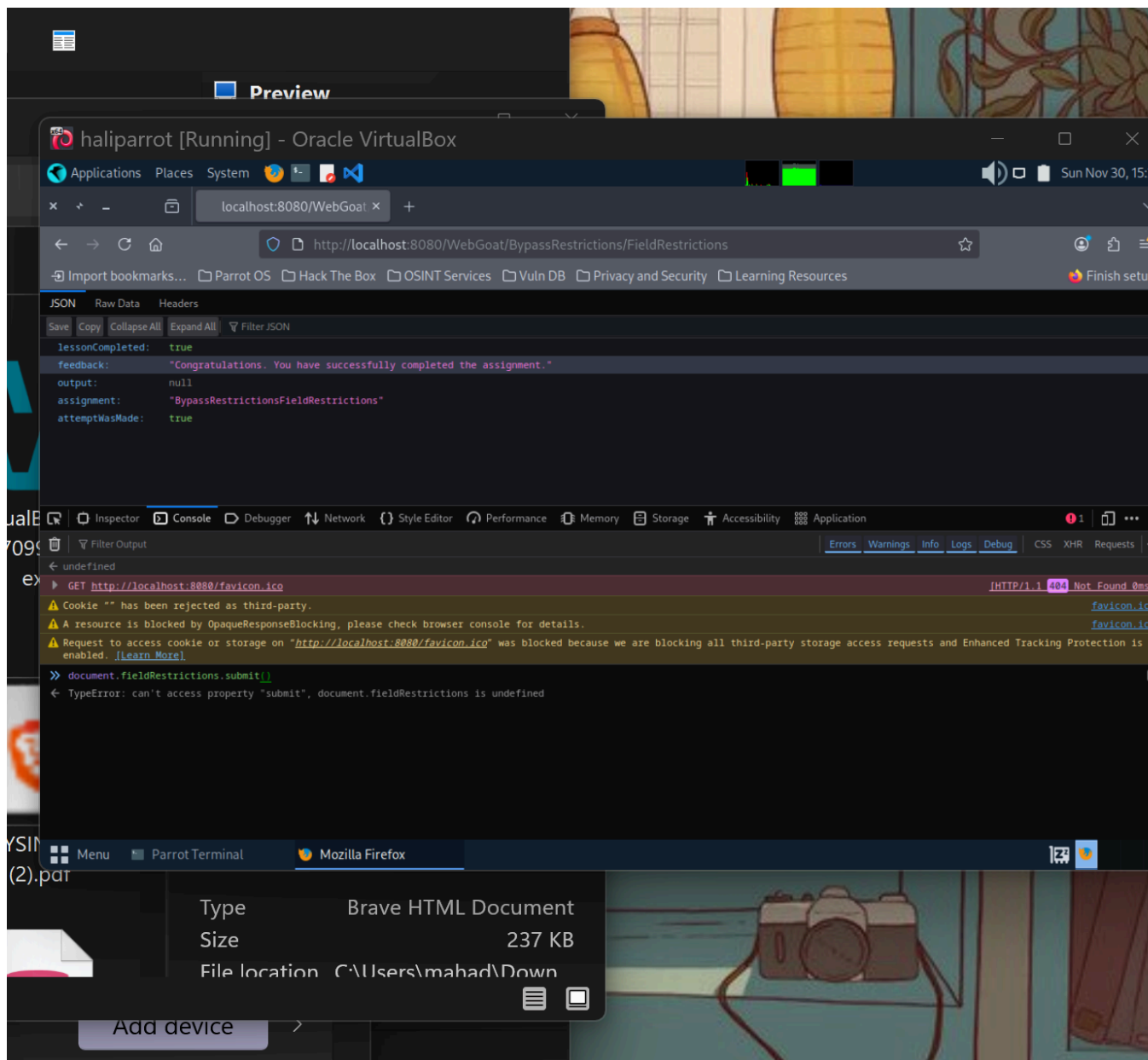
This screenshot shows "Option 1" in the Elements tab.

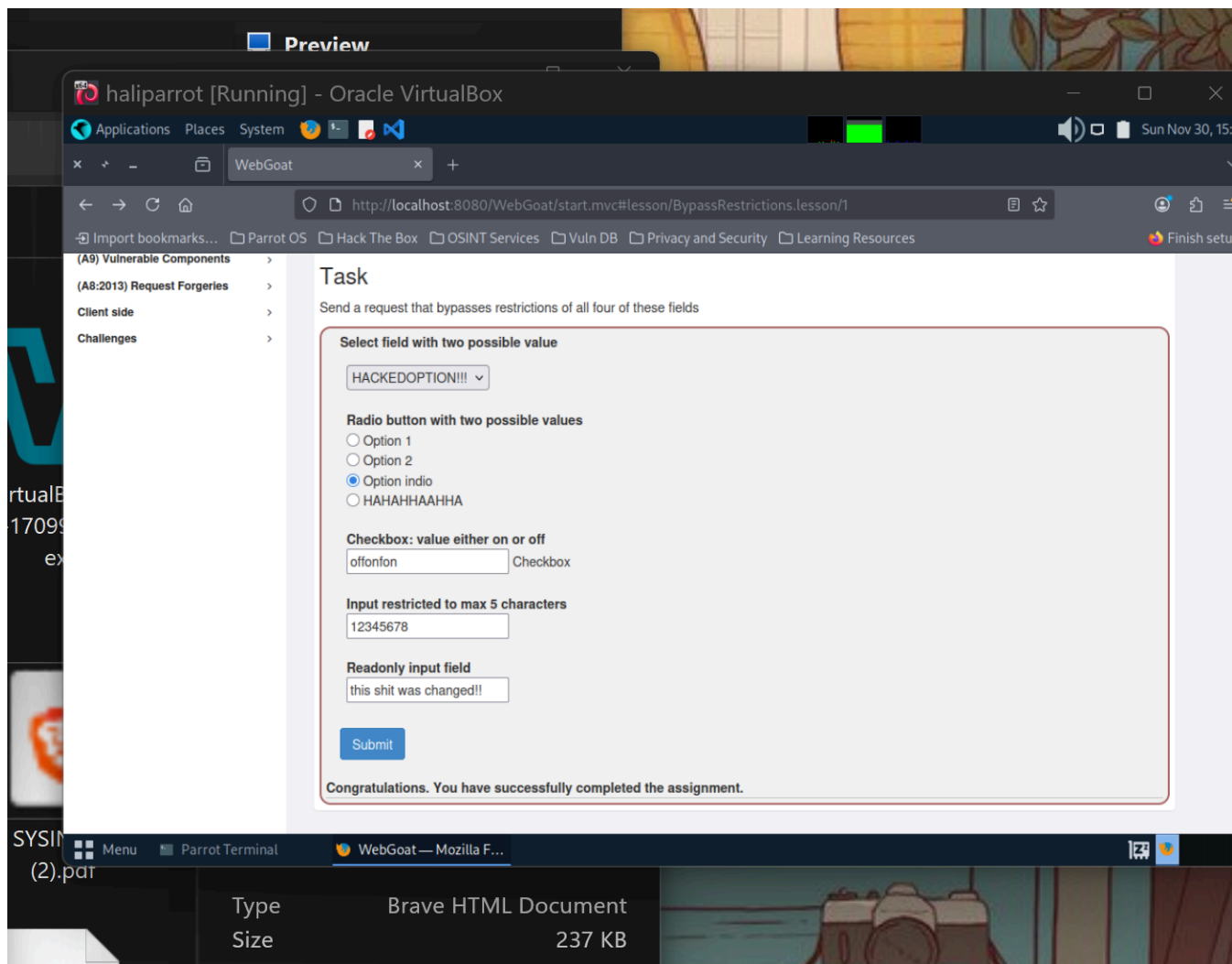
6. Alter each of the form fields such that they will allow input outside of the displayed validation rules.asd

HINT: You can right click on an element in the Elements tab and select "Edit as HTML"; this is useful to add an option in the dropdown or radio button elements.

HINT: You can double click on an attribute on an element in order to change the value; this might be useful to remove some restrictions.

HINT: The browser will ALWAYS send either "on" or "off" for a checkbox; think about how you might be able to change the type of the element.





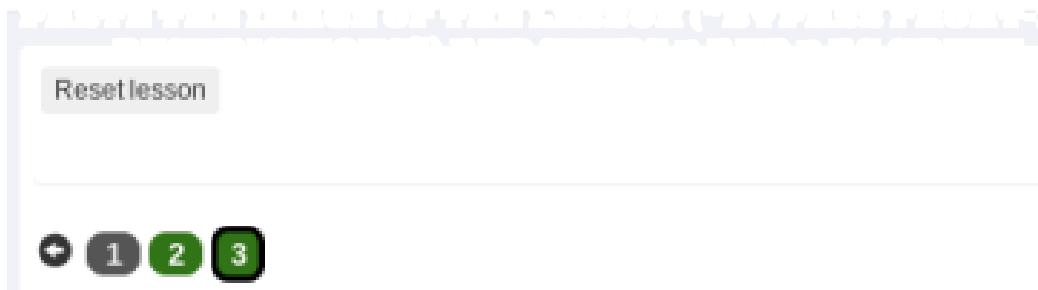
SECTION 2: Field restrictions (Step 3)

1. Log into WebGoat
2. From the left navigation bar, select "Client side"
3. From the left navigation bar, select "Bypass front-end restrictions"
4. Click on Step 3 (a red box)
5. The task is to alter what information is sent to the server. In Part 1, editing the HTML was sufficient because the restrictions were enforced in HTML. Part 2 cannot be completed that way because the validation is performed with JavaScript code run on the client. The key to this is to modify what is sent to the server.
6. Open up the developer tools; there are many ways to accomplish this (press F-12, or

- Control-Shift-J). Once the developer tools have been opened, select the "Network" tab. This allows you to observe all web traffic between your browser and the server.
7. Figure out which request needs to be altered. Usually you can submit the form as-is and see the request in the Network tab. After you identify the request name, you will find that typing part of the request name in the filter will filter out uninteresting requests.
 8. Select the request which was sent when you clicked the "Submit" button and observe the "Payload" and "Preview" tabs in the network request inspector. This gives you an idea of what you are dealing with.
 9. Now you can duplicate the request, alter it and send it to the server in order to bypass the JavaScript validation. Right click on the request, select "Copy" and then "Copy as fetch". This will put into your clipboard some JavaScript code which sends the request.
 10. Open the developer tools console. Either select the Console tab or press the "escape" key on your keyboard.
 11. Paste the JavaScript code into the console. DO NOT PRESS ENTER yet. Instead, alter the request by changing the field values to a value which would not be allowed.
 12. Press ENTER and the request will be sent.
 13. Go back to the Network tab in the developer tools and observe that another request has been sent. Inspect the Payload and Preview tabs. More importantly, notice that the red 3 has turned green, indicating success.

EVIDENCE #1

Bypass front-end restrictions



PART II: Client Side Filtering

Client side filtering refers to a vulnerability where secret information may be sent to the web browser. Casual users likely will not notice the extra information, but security through obscurity is not security at all.

SECTION I: Client side filtering (Steps 1 and 2)

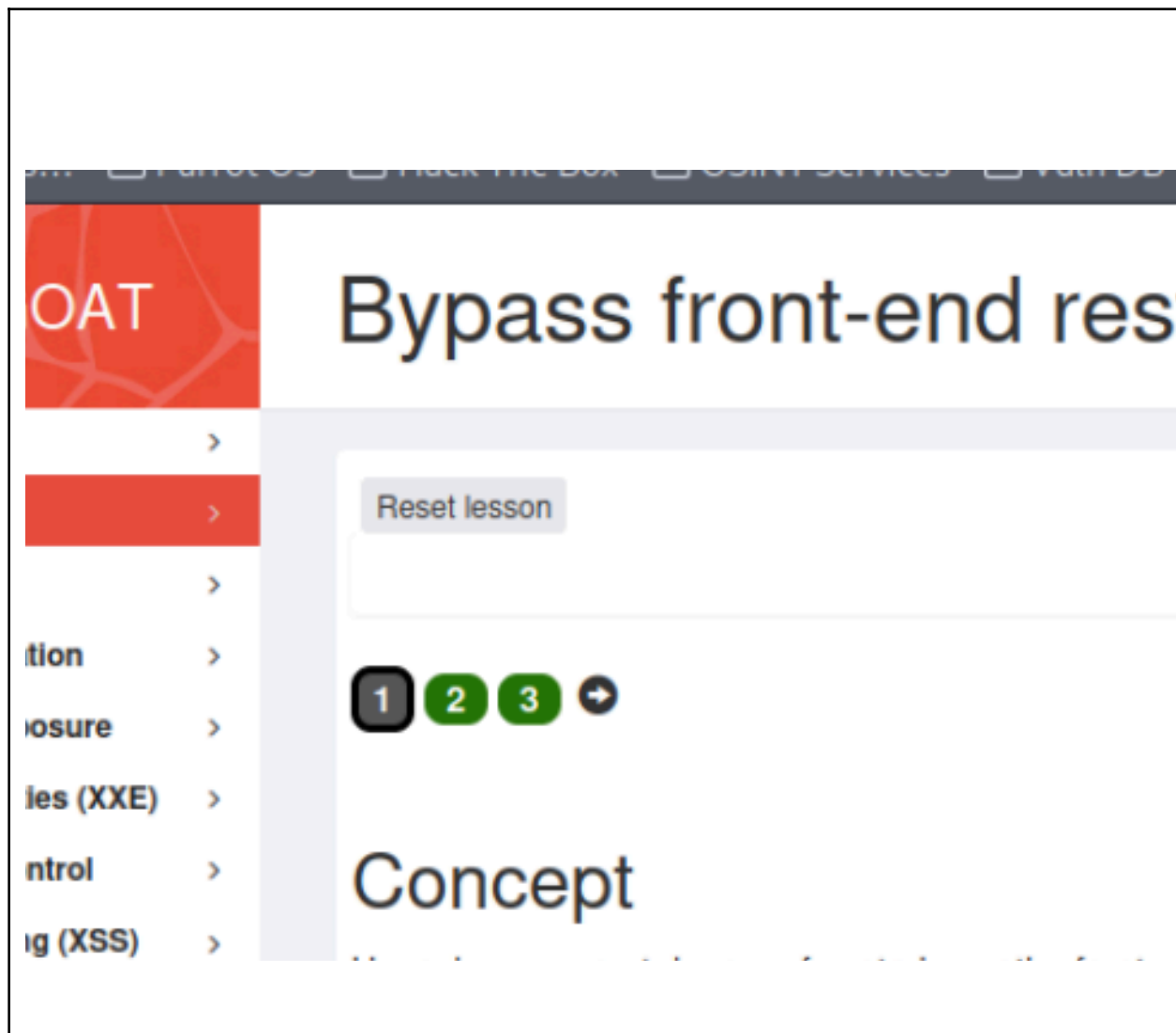
1. Log into WebGoat
2. From the left navigation bar, select "Client side"
3. From the left navigation bar, select "Client side filtering"
4. Click on Step 1 (a gray box) to learn about client side filtering
5. Click on Step 2 (a red box)
6. Complete the task by finding the CEO's salary

HINT: Because the information is being sent FROM the server TO the web browser, it is helpful to look at the network traffic. Keep an eye on the requests sent to the server when you interact with the website. Once the developer tools have been opened, select the "Network" tab. This allows you to observe all web traffic between your browser and the server.

SECTION 2: Name of part 2

1. Log into WebGoat
2. From the left navigation bar, select "Client side"
3. From the left navigation bar, select "Client side filtering"
4. Click on Step 3 (a red box)
5. Complete the task by buying a free phone

HINT: Because the information is being sent FROM the server TO the web browser, it is helpful to look at the network traffic. Keep an eye on the requests sent to the server when you interact with the website. Once the developer tools have been opened, select the "Network" tab. This allows you to observe all web traffic between your browser and the server.



PART III: HTML Tampering

HTML tampering is a technique which can take advantage of a poorly designed website by allowing users to modify critical data because the data resides on the client side and is not validated on the server.

SECTION I: Client side filtering (Steps 1, 2 and 3)

1. Log into WebGoat
2. From the left navigation bar, select "Client side"
3. From the left navigation bar, select "HTML tampering"
4. Click on Step 1 (a gray box) to learn about client side filtering

5. Click on Step 2 (a red box)

6. Complete the task buying an inexpensive TV

HINT: Look at the Network tab when you attempt to buy a TV. Inspecting the Payload tab for the request will show you the 2 parameters that are being sent to the server. It may be helpful to note that for a form being submitted to a server, each parameter sent has a corresponding form element whose "name" attribute has a value which corresponds to the parameter. For example, the payload should have a "QTY" parameter. Look at the Elements tab for a form element with the attribute "name" which has a value for "QTY".

HINT: Not all form fields are visible; these have an attribute "type" with a value of "hidden". Even though these values are not visible, they are still sent to the server.

HINT: Because this is "HTML tampering", you will need to alter the HTML. This can be done in the Elements tab. You need to identify which HTML to edit.

